

Machine Learning Zoomcamp

Módulo 2 — Regresión: predicción del precio de autos

Apuntes de las 16 clases

Elaborado a partir de las transcripciones de clase

Índice

2.1 — Proyecto: predicción del precio de autos	5
El problema	5
El dataset	5
Plan del proyecto	5
Dónde está el código	5
2.2 — Preparación de datos	6
Descargar y cargar el CSV	6
Problema 1: nombres de columnas inconsistentes	6
Problema 2: valores de texto inconsistentes	6
2.3 — Análisis exploratorio	8
Recorrer las columnas	8
Distribución del precio	8
Por qué la cola larga es un problema	9
Valores faltantes	9
Resumen	10
2.4 — Framework de validación	11
Calcular los tamaños	11
Slicing con iloc	11
Mezclar con un índice aleatorio	12
Reiniciar el índice	12
Separar el target	12
Borrar el target de los DataFrames	12
2.5 — Regresión lineal	14
Una observación = un vector	14
La fórmula	14
Implementación	14
Interpretación de los pesos	15
Deshacer el logaritmo	15
2.6 — Regresión lineal en forma vectorial	17
La sumatoria es un producto punto	17
Meter el bias dentro del producto punto	17
Generalizar a todos los autos	17
Resumen del recorrido	18
2.7 — Entrenamiento: ecuación normal	19

El problema	19
La solución aproximada	19
Paso a paso en NumPy	19
No olvidar el bias	20
La función final	20
2.8 — Modelo base (baseline)	21
Elegir las features	21
Valores faltantes	21
Predecir sobre train	21
Comparar visualmente	22
2.9 — RMSE (Root Mean Squared Error)	23
La fórmula	23
Ejemplo paso a paso	23
Implementación	23
2.10 — RMSE en el conjunto de validación	25
El problema	25
La función prepare_X	25
Entrenar y validar	25
2.11 — Feature engineering	26
De año a edad	26
Agregar la feature en prepare_X	26
Resultado	26
2.12 — Variables categóricas	28
Una categórica disfrazada: number_of_doors	28
One-hot encoding	28
Agregarlo a prepare_X	28
Agregar la marca (make)	29
Todas las categóricas	29
¡Se rompió!	29
2.13 — Regularización	31
El problema: la inversa de $X^T X$	31
Por qué en nuestro dataset no dio error	31
La solución: sumar un número a la diagonal	31
Implementación	32
λ es un parámetro	32

2.14 — Ajuste del modelo	33
Probar varios valores de r	33
Qué se observa	33
Entrenar con el r elegido	33
2.15 — Usar el modelo	34
Entrenar el modelo final	34
Usar el modelo para un auto nuevo	34
Resumen del flujo	35
2.16 — Resumen de la sesión 2	36
El proyecto (2.1)	36
Preparación de datos (2.2)	36
Análisis exploratorio (2.3)	36
Framework de validación (2.4)	36
Regresión lineal (2.5–2.6)	36
Ecuación normal (2.7)	36
Modelo base (2.8)	36
RMSE (2.9–2.10)	36
Feature engineering (2.11)	36
Variables categóricas (2.12)	37
Regularización (2.13)	37
Ajuste del modelo (2.14)	37
Usar el modelo (2.15)	37
Qué viene después	37

2.1 — Proyecto: predicción del precio de autos

El problema

Retomamos el escenario de la introducción: un sitio de **clasificados online** para vender autos. El usuario que publica su auto tiene que poner un precio y no siempre sabe cuánto pedir. Queremos un modelo que le **sugiera el mejor precio**.

El dataset

Usamos un dataset de precios de autos de **Kaggle**. Cada fila es un auto con sus características (features):

- marca (make), modelo, año
- motor, tipo de combustible, transmisión, etc.
- y el precio: la columna **MSRP**

MSRP = *Manufacturer Suggested Retail Price*, el precio de venta sugerido por el fabricante. Es nuestro **target**: lo que queremos predecir.

Idea general: a partir de las features del auto, predecir su precio.

Plan del proyecto

Paso	Qué se hace
1	Obtener los datos y hacer análisis exploratorio (EDA)
2	Preparar el dataset
3	Entrenar una regresión lineal para predecir el precio
4	Ver cómo funciona por dentro: implementarla nosotros mismos
5	Evaluar el modelo con RMSE (<i>root mean squared error</i>)
6	Feature engineering : crear nuevas features para el modelo
7	Resolver problemas de estabilidad numérica con regularización
8	Usar el modelo

Dónde está el código

Todo está en el repo del libro, **mlbookcamp-code**, en la carpeta de *chapter 2, car price*. Hay dos archivos:

- el **notebook** con todo el código de la sesión;
- el **CSV** con el dataset que usaremos para entrenar.

En clase se hace live coding y el notebook queda disponible.

El siguiente paso no es todavía el EDA: primero hay que **cargar y preparar** el CSV.

2.2 — Preparación de datos

Descargar y cargar el CSV

El dataset original está en Kaggle, pero hay una copia en el repo [mlbookcamp-code](#) (chapter 2, car price). Se copia la URL del archivo y se descarga con `wget` (o con «Guardar como» en el navegador).

```
import pandas as pd
import numpy as np

!wget <url-del-csv>          # en el notebook; o descargarlo a mano

df = pd.read_csv('data.csv')
df.head()                  # primeras 5 filas
```

Hábito del instructor: después de cargar, mirar siempre las primeras filas con `head()`.

Problema 1: nombres de columnas inconsistentes

Algunos nombres tienen mayúsculas, otros no; algunos usan `_` y otros espacios (Engine HP, Transmission Type, MSRP...).

Con espacios no se puede acceder con punto: `df.Transmission Type` no funciona, hay que usar `df['Transmission Type']`.

Solución: minúsculas y `_` en vez de espacios

`df.columns` es un **Index** de pandas. Igual que una Series, tiene el accesor `.str` para aplicar operaciones de texto a todos los elementos a la vez, y se pueden encadenar:

```
df.columns.str.lower()          # todo a minúsculas
df.columns.str.lower().str.replace(' ', '_')  # además, espacios → _

df.columns = df.columns.str.lower().str.replace(' ', '_')  # guardar el resultado
```

Problema 2: valores de texto inconsistentes

Lo mismo pasa con los **valores**: a veces en mayúsculas, a veces no. Hay que normalizarlos, pero `.str` solo funciona sobre columnas de texto, así que primero hay que encontrarlas.

Encontrar las columnas de texto

```
df.dtypes          # tipo de cada columna
```

dtype	Ejemplo	¿Nos interesa?
<code>int64</code>	year	No
<code>float64</code>	engine_hp	No
<code>object</code>	make, model	Sí

*Al leer un CSV, las columnas `object` son en la práctica **strings**.*

```
df.dtypes == 'object'          # Series de booleanos
df.dtypes[df.dtypes == 'object'] # solo las columnas object
df.dtypes[df.dtypes == 'object'].index # nos interesan los NOMBRES (el índice)

strings = list(df.dtypes[df.dtypes == 'object'].index)
```

Los valores de esa Series son todos «object», no aportan nada; lo útil es el **índice**, que contiene los nombres de las columnas. Pasarlo a lista es solo porque se ve más prolijo.

Aplicar la misma limpieza a cada columna de texto

```
for col in strings:
    df[col] = df[col].str.lower().str.replace(' ', '_')

df.head()
```

Resultado: todo en minúsculas y con `_` en vez de espacios, tanto en los nombres de las columnas como en los valores.

2.3 — Análisis exploratorio

Objetivo del **EDA** (*exploratory data analysis*): entender cómo se ven los datos, qué valores hay, y aprender más sobre los datos y el problema.

Recorrer las columnas

Para cada columna, mostrar algunos valores, los primeros valores únicos y cuántos valores únicos hay:

```
for col in df.columns:
    print(col)
    print(df[col].unique()[:5])    # primeros 5 valores únicos
    print(df[col].nunique())       # cantidad de valores únicos
    print()
```

Con `head()` se ven solo las primeras filas (todas BMW), poco informativo. Más útil es mirar los **valores únicos**.

Columna	Qué es	Observación
<code>make</code>	fabricante	48 valores (bmw, audi, fiat, mercedes-benz, chrysler...)
<code>model</code>	modelo	más granular: muchos más valores
<code>year</code>	año	numérica, 28 valores
<code>engine_fuel_type</code>	tipo de combustible	10 tipos
<code>engine_hp</code>	potencia del motor	
<code>engine_cylinders</code>	cantidad de cilindros	
<code>transmission_type,</code> <code>driven_wheels, ...</code>	otras características	
<code>highway_mpg</code>	millas por galón en carretera	
<code>city_mpg</code>	millas por galón en ciudad	
<code>popularity</code>	menciones en Twitter	los autores del dataset las contaron
<code>msrp</code>	precio	el target; muchísimos valores distintos

Distribución del precio

Para ver el panorama general hay que **graficar**; mirar números sueltos no alcanza.

```
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline    # para que los gráficos se vean en el notebook

sns.histplot(df.msrp, bins=50)
```

- **Matplotlib**: librería de gráficos de bajo nivel.
- **Seaborn**: construida sobre Matplotlib, hace las cosas más fáciles.
- `bins` = cantidad de barras del histograma.

En el eje, **1e6** es notación científica: 10^6 = un millón.

Long tail

Casi todos los autos son relativamente baratos y hay muy pocos carísimos (1, 1.5, 2 millones). Esa forma se llama **distribución de cola larga** (*long tail*).

Para ver mejor la zona donde están la mayoría, se hace zoom:

```
sns.histplot(df.msrp[df.msrp < 100000], bins=50)
```

Observaciones:

- Hay un **pico raro en 1000**: probablemente el precio mínimo que permite la plataforma.
- Hay muchos autos (unos 700) alrededor de 25 000, y después la cantidad va bajando.

Las colas largas son muy comunes en precios: la mayoría de las cosas son accesibles para el público general y unas pocas son muy caras, porque poca gente puede pagarlas.

Por qué la cola larga es un problema

Una distribución así **no es buena para ML**: la cola confunde al modelo. Hay que eliminar ese efecto, y lo habitual es aplicar el **logaritmo** al precio: los valores grandes quedan mucho más comprimidos.

```
np.log([1, 10, 1000, 100000]) # crece mucho más despacio que los valores
```

log1p: logaritmo de (x + 1)

log(0) no existe. En este dataset no hay precios 0 (el mínimo es 1000), pero es práctica común **sumar 1** antes de aplicar el logaritmo: así **log(0 + 1) = 0** y no hay problemas.

NumPy tiene un atajo que hace las dos cosas (**1p** = *plus one*):

```
np.log1p([0, 1, 10, 1000, 100000])  
# es lo mismo que  
np.log([0 + 1, 1 + 1, 10 + 1, 1000 + 1, 100000 + 1])
```

Aplicado al precio:

```
price_logs = np.log1p(df.msrp)  
  
sns.histplot(price_logs, bins=50)
```

Resultado: **la cola desaparece**. Los precios muy altos colapsan hacia una zona compacta y la forma se parece a una **campana**: la **distribución normal**.

Si el target se parece a una normal (un centro claro y que baja hacia ambos lados), los modelos predicen mucho mejor que con una cola larga. El pico en 1000 sigue ahí, pero igual se parece bastante más a una normal.

Valores faltantes

En `number_of_doors`, por ejemplo, aparece **NaN** (*not a number*): en pandas indica que el valor **falta**, no se registró.

```
df.isnull()          # True/False celda por celda (poco útil así)
df.isnull().sum()    # cantidad de valores faltantes por columna
```

Hay varios: para algunos autos no se conoce el tipo de combustible, la categoría de mercado (`market_category`), los caballos de fuerza o los cilindros.

Hay que tenerlo en cuenta: **antes de entrenar habrá que hacer algo con los valores faltantes.**

Resumen

1. Recorrimos las columnas y sus valores para entender los datos.
2. El precio tiene **cola larga** → la eliminamos con `np.log1p`.
3. Hay **valores faltantes** que habrá que tratar antes de entrenar.

Próximo paso: armar el **framework de validación**.

2.4 — Framework de validación

Como vimos en 1.5 — Selección de modelos, para validar hay que dividir el dataset en **tres partes**:

train 60 %	val 20 %	test 20 %
------------	----------	-----------

- **train**: para entrenar.
- **validation**: para comprobar si el modelo funciona bien.
- **test**: se deja para el final; se usa muy de vez en cuando, solo para confirmar que el modelo anda bien.

De cada parte se obtiene una matriz de features **x** y un target **y**: `X_train`, `y_train`, `X_val`, `y_val`, `X_test`, `y_test`.

En esta clase se hace todo **a mano**, con pandas y NumPy, sin librerías extra.

Calcular los tamaños

El dataset tiene casi 12 000 filas → el 20 % son unas 2 400. Los tamaños tienen que ser **enteros**, así que se convierten con `int`:

```
n = len(df)

n_val = int(n * 0.2)
n_test = int(n * 0.2)
n_train = int(n * 0.6)

n_val + n_test + n_train == n    # False: por el redondeo no suman n
```

Por el redondeo, la suma queda ****por debajo de n**** y se perderían algunas filas. La solución: train se queda con todo lo que sobra.

```
n_train = n - n_val - n_test

n_val, n_test, n_train    # (2382, 2382, 7150)
```

Slicing con `iloc`

`iloc` acepta rangos (el límite superior es **exclusivo**):

```
df.iloc[:10]    # filas 0 a 9
df.iloc[10:20]  # filas 10 a 19
df.iloc[10:]    # desde la 10 hasta el final
```

Primer intento (secuencial):

```
df_train = df.iloc[:n_train]
df_val = df.iloc[n_train:n_train + n_val]
df_test = df.iloc[n_train + n_val:]
```

En clase primero se arma con validation al principio y después se reordena a train → val → test, que es más lógico.

Problema: los datos están ordenados

Si se corta en orden, por ejemplo todos los BMW pueden caer en un solo conjunto y no aparecer en train. Por eso hay que **mezclar** (shuffle).

En general siempre conviene mezclar los datos, por si existe algún orden accidental que haya que romper.

Mezclar con un índice aleatorio

`iloc` también acepta una secuencia arbitraria de posiciones y devuelve las filas en ese orden. Entonces: generar `0 ... n-1`, mezclarlo y usarlo para indexar.

```
idx = np.arange(n)
np.random.shuffle(idx)

df.iloc[idx[:10]]      # las primeras 10 filas en orden mezclado
```

Reproducibilidad: semilla

Sin semilla, cada ejecución da una mezcla distinta. Con una semilla fija, el resultado es el mismo (con la misma versión de NumPy):

```
np.random.seed(2)

idx = np.arange(n)
np.random.shuffle(idx)

df_train = df.iloc[idx[:n_train]]
df_val = df.iloc[idx[n_train:n_train + n_val]]
df_test = df.iloc[idx[n_train + n_val:]]

len(df_train), len(df_val), len(df_test)
```

Reiniciar el índice

Después de mezclar, cada DataFrame conserva el índice original (desordenado). No lo necesitamos, así que se resetea y se descarta:

```
df_train = df_train.reset_index(drop=True)
df_val = df_val.reset_index(drop=True)
df_test = df_test.reset_index(drop=True)
```

Separar el target

Se aplica la transformación logarítmica al precio y se toma directamente el **array de NumPy** con `.values` (no hacen falta el índice ni el resto de la Series):

```
y_train = np.log1p(df_train.msrp.values)
y_val = np.log1p(df_val.msrp.values)
y_test = np.log1p(df_test.msrp.values)
```

Borrar el target de los DataFrames

```
del df_train['msrp']  
del df_val['msrp']  
del df_test['msrp']
```

Consejo del instructor: siempre borrar el target después de separarlo. Si queda en el DataFrame, se puede usar por accidente como feature para predecir el precio: el modelo sale «perfecto» y uno pierde mucho tiempo buscando qué anda mal. Dice que le pasó muchas veces.

Con esto los datos quedan listos para entrenar. Próximo paso: **regresión lineal**.

2.5 — Regresión lineal

La regresión lineal es un modelo para problemas de **regresión**: predice un **número** (en nuestro caso, el precio de un auto).

Recordando la fórmula de la introducción:

$$g(X) \approx y$$

- $g \rightarrow$ el modelo (aquí, regresión lineal)
- $X \rightarrow$ matriz de features
- $y \rightarrow$ target (el precio)

Antes de trabajar con la matriz completa, la clase mira el caso simplificado: **una sola observación** (un auto).

Una observación = un vector

Un auto es una fila de la matriz de features, o sea un vector de n elementos:

$$x_i = (x_{i1}, x_{i2}, \dots, x_{in})$$

Queremos una función $g(x_i)$ que tome esas características y devuelva algo **cercano al precio y_i** .

Ejemplo de la clase

Se toma la fila 10 del **dataset de entrenamiento** (solo train, nada de validación ni test): un Rolls-Royce Phantom Drophead Coupe de 2015. Para simplificar se usan solo 3 features:

Feature	Valor
engine_hp	453
city_mpg	11
popularity	86

```
xi = [453, 11, 86]
```

```
def g(xi):  
    # hacer algo  
    return 10000    # por ahora, una predicción inventada
```

La fórmula

$$g(x_i) = w_0 + w_1 \cdot x_{i1} + w_2 \cdot x_{i2} + w_3 \cdot x_{i3}$$

- $w_0 \rightarrow$ **bias term** (término de sesgo): la predicción que haríamos **sin saber nada del auto**.
- $w_1, w_2, w_3 \rightarrow$ **pesos** (weights): cada feature se multiplica por su peso.

Escrita de forma compacta con una sumatoria (se usa j porque i ya indica el auto):

$$g(x_i) = w_0 + \sum_j w_j \cdot x_{ij} \quad (j \text{ de } 1 \text{ a } n)$$

Implementación

En Python los índices van de 0 a $n-1$, no de 1 a n .

```
w0 = 0
w = [1, 1, 1]

def linear_regression(xi):
    n = len(xi)          # debe coincidir con len(w)

    pred = w0
    for j in range(n):
        pred = pred + w[j] * xi[j]

    return pred
```

El **for** es la sumatoria: cada feature se multiplica por su peso y se va acumulando en la predicción.

Con pesos inventados (de dónde salen se ve más adelante):

```
w0 = 7.17
w = [0.01, 0.04, 0.002]

linear_regression(xi)    # → 12.31
```

```
7.17 + 453·0.01 + 11·0.04 + 86·0.002 = 12.31
```

Interpretación de los pesos

Término	Peso	Lectura
bias w_0	7.17	Predicción para un "auto promedio" del que no sabemos nada
engine_hp	0.01	Más caballos de fuerza → más caro (100 hp suman 1)
city_mpg	0.04	Cada milla por galón extra sube un poco el precio
popularity	0.002	Peso muy bajo: hacen falta muchísimas menciones en Twitter para que pese

*El profesor comenta que el signo de **city_mpg** puede tener sentido según cómo se mire (autos que consumen más suelen ser más "fancy"), y que la popularidad casi no afecta salvo en casos extremos (¿Tesla?).*

Deshacer el logaritmo

El valor **12.31 no es el precio**: el target se transformó con $\log(y + 1)$. Para volver a la escala original hay que aplicar la exponencial y restar 1:

```
np.exp(12.31) - 1    # → ~222.000 dólares
```

Igual que existe un atajo para el logaritmo (**np.log1p**), existe uno para su inversa:

```
np.expm1(12.31)     # equivale a np.exp(x) - 1
```

`np.log1p` y `np.exp1` se deshacen mutuamente.

Hasta aquí: la fórmula aplicada a **un solo vector** de tamaño 3. En la siguiente lección se generaliza a muchos ejemplos.

2.6 — Regresión lineal en forma vectorial

Objetivo: pasar de la fórmula para **un auto** a la forma con la **matriz completa X**.

La sumatoria es un producto punto

La fórmula para un auto:

$$g(x_i) = w_0 + \sum_j w_j \cdot x_{ij}$$

La sumatoria no es otra cosa que un **producto punto** (vector-vector) entre el vector de features y el vector de pesos:

$$g(x_i) = w_0 + x_i^T w$$

Implementación

Se reutiliza la multiplicación vector-vector del módulo 1:

```
def dot(xi, w):
    n = len(xi)

    res = 0.0
    for j in range(n):
        res = res + xi[j] * w[j]

    return res

def linear_regression(xi):
    return w0 + dot(xi, w)
```

Con NumPy no hace falta escribir `dot`: basta con `xi.dot(w)`.

Meter el bias dentro del producto punto

El w_0 queda "colgando" solo, sin ningún x . Truco: imaginar una **feature ficticia** x_{i0} que **siempre vale 1**.

```
w = (w0, w1, w2, ..., wn)      → n+1 elementos
xi = (1, xi1, xi2, ..., xin)    → n+1 elementos
```

Al hacer el producto punto, $w_0 \cdot 1 = w_0$, y el resto es igual que antes:

$$g(x_i) = x_i^T w \quad (\text{con el 1 al inicio de } x_i \text{ y } w_0 \text{ al inicio de } w)$$

```
w_new = [w0] + w                # con listas, + concatena: antepone w0

def linear_regression(xi):
    xi = [1] + xi                # antepone el 1
    return dot(xi, w_new)
```

El resultado es el mismo que con la versión anterior.

Generalizar a todos los autos

La matriz X ahora tiene una **columna de unos** al principio:

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \quad W = \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_n \end{bmatrix}$$

- X es de $m \times (n+1)$: **m filas** (autos) y **n+1 columnas**.
- Para cada fila se hace el producto punto con $w \rightarrow$ una predicción por auto.

$$\hat{y} = (x_1^T w, x_2^T w, \dots, x_m^T w) = X \cdot w$$

Esto es exactamente una **multiplicación matriz-vector**. Aplicar la regresión lineal = multiplicar X por w .

Implementación

```
# valores de ejemplo inventados en clase (con el 1 al inicio)
x1 = [1, 148, 24, 1385]
x2 = [1, 132, 25, 2031]
x10 = [1, 453, 11, 86]

X = [x1, x2, x10]      # lista de listas
X = np.array(X)        # → matriz (array 2D de NumPy)

def linear_regression(X):
    return X.dot(w_new)

linear_regression(X)    # una predicción por cada auto
```

Resumen del recorrido

Paso	Forma
1	for sobre las features (sumatoria)
2	Producto punto $w_0 + x_i^T w$
3	Feature ficticia = 1 $\rightarrow x_i^T w$
4	Todos los autos a la vez $\rightarrow X \cdot w$

Queda la pregunta: **¿de dónde salen los pesos w ?** $\rightarrow$ siguiente lección.

2.7 — Entrenamiento: ecuación normal

Ya sabemos **aplicar** el modelo ($X \cdot w$). Ahora: ¿cómo se **obtienen** los pesos w ?

El problema

Queremos que las predicciones se parezcan al target:

$$g(X) = X \cdot w \approx y$$

Idealmente $X \cdot w = y$: un sistema de ecuaciones a resolver para w .

Si X tuviera inversa

$$\begin{aligned} X^{-1} \cdot X \cdot w &= X^{-1} \cdot y \\ I \cdot w &= X^{-1} \cdot y \\ w &= X^{-1} \cdot y \end{aligned}$$

Pero X es **rectangular** ($m \times (n+1)$): muchas filas, pocas columnas), así que **no tiene inversa**. El sistema no tiene solución exacta.

La solución aproximada

Se multiplican ambos lados por X^T :

$$X^T X \cdot w = X^T y$$

$X^T X$ es la **matriz de Gram**. Siempre es **cuadrada**, de $(n+1) \times (n+1)$, así que su inversa **puede** existir (normalmente existe; no siempre — se retoma más adelante).

Multiplicando por $(X^T X)^{-1}$:

$$\begin{aligned} (X^T X)^{-1} \cdot X^T X \cdot w &= (X^T X)^{-1} \cdot X^T y \\ I \cdot w &= (X^T X)^{-1} \cdot X^T y \\ w &= (X^T X)^{-1} \cdot X^T y && \leftarrow \text{ecuación normal} \end{aligned}$$

Este w **no es** la solución del sistema (no existe), pero es la **más cercana posible**. La demostración requiere bastante matemática; el profesor recomienda el libro *The Elements of Statistical Learning* para quien quiera ver la derivación. Lo de la clase es a nivel intuitivo.

Paso a paso en NumPy

Se inventa una matriz X con **más filas que columnas** (con pocas filas la inversa probablemente no existiría) y un y cualquiera.

```
XTX = X.T.dot(X)           # matriz de Gram
XTX_inv = np.linalg.inv(XTX) # su inversa

XTX.dot(XTX_inv)           # ≈ matriz identidad
```

La comprobación no da exactamente la identidad: aparecen números diminutos fuera de la diagonal. Es por la **precisión finita** de la máquina; se pueden tratar como cero.

```
w = XTX_inv.dot(X.T).dot(y)
```

No olvidar el bias

Si X no tiene la columna de unos, el modelo se entrena **sin bias term**. A veces eso tiene sentido, pero queremos el bias porque es la **línea base**: cuánto costaría un auto del que no sabemos nada.

Para agregar la columna de unos:

```
ones = np.ones(X.shape[0])    # un 1 por cada fila
X = np.column_stack([ones, X]) # apila como columnas: los unos quedan primero
```

Luego se repite la ecuación normal y se separa el resultado:

```
w_full = XTX_inv.dot(X.T).dot(y)

w0 = w_full[0]    # bias
w = w_full[1:]    # resto de los pesos
```

Pesos negativos

En el ejemplo algunos pesos salen **negativos**: significa que la feature **resta** al precio en vez de sumar. Ejemplo de la clase: si la feature fuera la **edad** del auto, cada año extra bajaría el precio.

La función final

```
def train_linear_regression(X, y):
    ones = np.ones(X.shape[0])
    X = np.column_stack([ones, X])

    XTX = X.T.dot(X)
    XTX_inv = np.linalg.inv(XTX)
    w_full = XTX_inv.dot(X.T).dot(y)

    return w_full[0], w_full[1:]
```

*Los unos se agregan **dentro** de la función: como usuarios no hay que preocuparse de añadirlos a la matriz de features.*

```
w0, w = train_linear_regression(X, y)
```

2.8 — Modelo base (baseline)

Se usa `train_linear_regression` de la lección anterior para construir un primer modelo de precios: el **baseline**.

Elegir las features

Para el baseline se usan **solo las columnas numéricas** (se revisan con `df.dtypes`):

Columna	Significado
<code>engine_hp</code>	caballos de fuerza del motor
<code>engine_cylinders</code>	número de cilindros
<code>highway_mpg</code>	millas por galón en carretera
<code>city_mpg</code>	millas por galón en ciudad
<code>popularity</code>	popularidad (menciones en Twitter)

```
base = ['engine_hp', 'engine_cylinders', 'highway_mpg',  
        'city_mpg', 'popularity']  
  
X_train = df_train[base].values    # .values → array de NumPy
```

Valores faltantes

Al entrenar aparecen **NaN** en el resultado: hay **missing values** en `engine_hp` y `engine_cylinders`.

```
df_train[base].isnull().sum()
```

Lo más simple es rellenarlos con cero:

```
X_train = df_train[base].fillna(0).values  
  
w0, w = train_linear_regression(X_train, y_train)
```

¿Qué significa rellenar con 0?

Con dos features, si x_{i1} falta y se reemplaza por 0:

$$\begin{aligned} g(x_i) &= w_0 + x_{i1} \cdot w_1 + x_{i2} \cdot w_2 \\ &= w_0 + 0 \cdot w_1 + x_{i2} \cdot w_2 \\ &= w_0 + x_{i2} \cdot w_2 \end{aligned}$$

En la práctica, el modelo **ignora** esa feature para ese auto.

*Desde el sentido común, 0 no tiene mucho sentido: no existen autos con 0 caballos de fuerza ni motores con 0 cilindros. Pero en machine learning, en la práctica, **a veces 0 funciona bien**. Rellenar con la media (como en el homework) es posible, solo complica un poco el proceso.*

Predecir sobre train

Por ahora se aplica el modelo al mismo dataset de entrenamiento:

```
y_pred = w0 + X_train.dot(w)    # sin olvidar el bias
```

Comparar visualmente

Histograma de predicciones vs. target con Seaborn:

```
sns.histplot(y_pred, color='red', alpha=0.5, bins=50)
sns.histplot(y_train, color='blue', alpha=0.5, bins=50)
```

- **color** → distinguir las dos distribuciones.
- **bins=50** → menos barras.
- **alpha** → transparencia, para ver ambas superpuestas.

Lo que se observa: la distribución de las predicciones está corrida; incluso los **picos no coinciden**. En muchos casos el modelo predice **valores más bajos** que los reales.

*Mirar un gráfico sugiere que el modelo no es ideal, pero hace falta una forma **objetiva** de decir si un modelo es bueno, y de confirmar que una mejora es realmente una mejora. Para eso: **RMSE**, en la siguiente lección.*

2.9 — RMSE (Root Mean Squared Error)

En la lección anterior entrenamos el modelo base (solo variables numéricas) y graficamos predicciones vs. valores reales: se veía que estaban algo desfasados, pero **no teníamos un número** para medir qué tan malo es el modelo. El RMSE es una forma de evaluar modelos de regresión.

La fórmula

$$\text{RMSE} = \sqrt{\left(\frac{1}{m} \right) \cdot \sum (g(x_i) - y_i)^2} \quad \text{con } i = 1 \dots m$$

Parte	Qué es
$g(x_i)$	la predicción para la observación i
y_i	el valor real (el precio) de la observación i
$(g(x_i) - y_i)^2$	el error al cuadrado (squared error)
$(1/m) \cdot \sum \dots$	el promedio sobre las m observaciones → MSE
$\sqrt{\dots}$	la raíz cuadrada → RMSE

Ejemplo paso a paso

	Obs. 1	Obs. 2	Obs. 3	Obs. 4
Predicción (y_{pred})	10	9	11	10
Real (y_{train})	9	9	10.5	11.5
Diferencia	1	0	0.5	-1.5
Diferencia ²	1	0	0.25	2.25

$$\begin{aligned} \text{MSE} &= (1 + 0 + 0.25 + 2.25) / 4 = 0.875 \\ \text{RMSE} &= \sqrt{0.875} \approx 0.93 \end{aligned}$$

Implementación

```
def rmse(y, y_pred):
    error = y - y_pred
    se = error ** 2      # squared error
    mse = se.mean()     # mean squared error
    return np.sqrt(mse) # root mean squared error
```

No hace falta sumar y dividir por la cantidad de elementos: en NumPy alcanza con el método `.mean()`.

Versión simplificada (sin la variable intermedia `error`):

```
def rmse(y, y_pred):
    se = (y - y_pred) ** 2
    mse = se.mean()
    return np.sqrt(mse)
```

Se usa con las predicciones que ya teníamos:

```
rmse(y_train, y_pred)
```

Cuanto **más bajo** el RMSE, mejor el modelo. En la próxima lección se aplica sobre el conjunto de validación.

2.10 — RMSE en el conjunto de validación

El problema

Hasta ahora entrenamos el modelo base (5 variables numéricas) con el conjunto de **entrenamiento** y calculamos el RMSE **sobre ese mismo conjunto**.

Pero el esquema de validación es otro: el dataset se divide en tres partes (train / validation / test). Lo correcto es:

1. Entrenar con **train**.
2. Aplicar el modelo a **validation**.
3. Medir el RMSE sobre **validation**.

La función `prepare_X`

La línea que armaba la matriz de features hacía muchas cosas a la vez. Se separa en una función que sirve para **cualquier** dataset:

```
base = ['engine_hp', 'engine_cylinders', 'highway_mpg',
        'city_mpg', 'popularity']

def prepare_X(df):
    df_num = df[base]           # 1. seleccionar las columnas numéricas
    df_num = df_num.fillna(0)   # 2. rellenar los valores faltantes
    X = df_num.values           # 3. extraer la matriz (array de NumPy)
    return X
```

De una línea pasamos a cinco, pero es más fácil entender qué está pasando.

La idea clave: **preparar los datos exactamente igual** sea train, validation o test.

Entrenar y validar

```
# parte de entrenamiento: solo se toca df_train
X_train = prepare_X(df_train)
w0, w = train_linear_regression(X_train, y_train)

# parte de validación: misma preparación, mismo modelo
X_val = prepare_X(df_val)
y_pred = w0 + X_val.dot(w)

rmse(y_val, y_pred)
```

Parte	Qué hace
Entrenamiento	<code>prepare_X(df_train)</code> → entrenar → obtener <code>w0, w</code>
Validación	<code>prepare_X(df_val)</code> → aplicar los pesos aprendidos → <code>rmse(y_val, y_pred)</code>

Con esto ya tenemos una forma de evaluar el modelo sobre datos que no vio. El siguiente paso es **mejorarlo**.

2.11 — Feature engineering

Objetivo: agregar más features al modelo.

De año a edad

La columna **year** es de las más importantes para predecir el precio: un auto viejo suele ser más barato, uno nuevo más caro.

En vez de usar el año tal cual, se calcula la **edad** del auto. Para eso hay que saber cuándo se recolectaron los datos: en **2017**.

```
df_train['age'] = 2017 - df_train.year
```

Hay autos de 0 años, otros de 9, etc.

Agregar la feature en `prepare_X`

```
def prepare_X(df):
    df = df.copy()          # ¡no modificar el DataFrame original!
    df['age'] = 2017 - df.year

    features = base + ['age'] # features base + la nueva

    df_num = df[features]
    df_num = df_num.fillna(0)
    X = df_num.values
    return X
```

¿Por qué `df.copy()`?

Sin la copia, la línea `df['age'] = ...` **agrega la columna al DataFrame que le pasamos** (por ejemplo `df_train`).

Como usuario de la función, no quiero que cambie mis datos: podría hacer algo que después no se puede deshacer. Mejor trabajar sobre una copia.

Con `df.copy()`, la función modifica la copia, y al salir esa copia se descarta: lo único que importa es la `X` que devuelve. El `X_train` resultante tiene ahora **6 columnas**, la última es `age`.

Resultado

```
X_train = prepare_X(df_train)
w0, w = train_linear_regression(X_train, y_train)

X_val = prepare_X(df_val)
y_pred = w0 + X_val.dot(w)
rmse(y_val, y_pred)
```

Modelo	RMSE (validación)
Base (5 numéricas)	0.76
Base + age	0.51

Una mejora grande. Se confirma graficando predicciones vs. valores reales (ahora con `y_val`, porque se predice sobre validación): la forma de la distribución se parece mucho más, aunque todavía hay zonas

donde falla.

Hay mucho margen de mejora. Lo siguiente: **variables categóricas** (marca, modelo, etc.).

2.12 — Variables categóricas

Son variables cuyos valores son **categorías**, normalmente strings: marca (**make**), modelo, tipo de combustible, tipo de transmisión, tracción, etc.

```
df_train.dtypes    # las columnas de tipo object son categóricas
```

Una categórica disfrazada: `number_of_doors`

Parece numérica, pero en realidad es categórica: autos de 2, 3 y 4 puertas son **tipos distintos** de auto. Pandas la trata como número solo porque sus valores son números.

Puede ser importante: un auto de 2 puertas probablemente sea más caro que uno de 4.

One-hot encoding

Una columna categórica se representa con **varias columnas binarias**, una por valor:

number_of_doors	num_doors_2	num_doors_3	num_doors_4
2	1	0	0
3	0	1	0
4	0	0	1
2	1	0	0

En Pandas, con una comparación `==` y pasando el booleano a entero:

```
(df_train.number_of_doors == 2).astype('int')    # True/False → 1/0
```

En vez de escribir una línea por cada valor, un loop con un template de string:

```
for v in [2, 3, 4]:
    df_train['num_doors_%s' % v] = (df_train.number_of_doors == v).astype('int')
```

`%s` se reemplaza por el valor de `v`.

Agregarlo a `prepare_X`

```
def prepare_X(df):
    df = df.copy()
    features = base.copy()

    df['age'] = 2017 - df.year
    features.append('age')

    for v in [2, 3, 4]:
        df['num_doors_%s' % v] = (df.number_of_doors == v).astype('int')
        features.append('num_doors_%s' % v)

    df_num = df[features]
    df_num = df_num.fillna(0)
    X = df_num.values
    return X
```

Si se hace `append` directamente sobre `base`, cada llamada a la función le agrega `age` (y las demás) **otra vez** a la lista global. Por eso se trabaja sobre una copia (`features = base.copy()`), igual que con el `DataFrame`.

Resultado: la mejora es **casi despreciable**. El número de puertas no aporta mucho.

Agregar la marca (`make`)

`make` tiene muchos valores distintos (`df.make.nunique()`), así que se toman solo los más populares:

```
df.make.value_counts().head()          # top 5 marcas con su frecuencia

makes = list(df.make.value_counts().head().index)
# ['chevrolet', 'ford', 'volkswagen', 'toyota', 'dodge']
```

`value_counts()` devuelve los valores en el **índice**; con `.index` se obtienen y se envuelven en una lista de Python.

```
for v in makes:
    df['make_%s' % v] = (df.make == v).astype('int')
    features.append('make_%s' % v)
```

Se agregan 5 columnas nuevas. El RMSE baja alrededor de un 1 %: no tan drástico como con `age`, pero está bien.

Todas las categóricas

Mirando de nuevo `dtypes`: `make`, `engine_fuel_type`, `transmission_type`, `driven_wheels`, `market_category`, `vehicle_size`, `vehicle_style`.

El modelo (`model`) **no** se incluye: tiene demasiados valores.

Primero, un diccionario con los 5 valores más comunes de cada una:

```
categorical_variables = [
    'make', 'engine_fuel_type', 'transmission_type', 'driven_wheels',
    'market_category', 'vehicle_size', 'vehicle_style'
]

categories = {}

for c in categorical_variables:
    categories[c] = list(df[c].value_counts().head().index)
```

Después, dos loops anidados en `prepare_X` (sobre los pares clave–valor con `items()`):

```
for c, values in categories.items():
    for v in values:
        df['%s_%s' % (c, v)] = (df[c] == v).astype('int')
        features.append('%s_%s' % (c, v))
```

El primer `%s` se reemplaza por `c` (la variable) y el segundo por `v` (el valor).

¡Se rompió!

Modelo	RMSE (validación)
Base + age	≈ 0.51
+ todas las categóricas	≈ 41

Las predicciones son números enormes (negativos, en notación científica) y algunos pesos **w** también son gigantes.

Queríamos mejorar el modelo agregando variables y lo empeoramos. En la próxima lección se ve por qué pasa y cómo arreglarlo.

2.13 — Regularización

Al agregar las variables categóricas (2.12) el RMSE se disparó y los pesos salieron enormes. En esta clase se ve **por qué pasa** y cómo arreglarlo.

El problema: la inversa de $X^T X$

Recordemos la ecuación normal:

$$w = (X^T X)^{-1} X^T y$$

La parte delicada es invertir la **matriz de Gram $X^T X$** . Esa inversa no siempre existe. Ocurre típicamente cuando la matriz de features tiene **columnas duplicadas**: en álgebra lineal se dice que una columna es **combinación lineal** de las otras.

```
X = [
    [4, 4, 4],
    [3, 5, 5],
    [5, 1, 1],
    [5, 4, 4],
    [7, 5, 5],
    [4, 5, 5],
]
X = np.array(X)

XTX = X.T.dot(X)
np.linalg.inv(XTX)    # LinAlgError: Singular matrix
```

La segunda y la tercera columna son iguales → la matriz es **singular** y NumPy se niega a invertirla.

Por qué en nuestro dataset no dio error

Los datos reales tienen **ruido**. Basta con que un valor sea **5.0000001** en vez de **5** para que las columnas ya no sean *exactamente* iguales:

```
X = [
    [4, 4, 4],
    [3, 5, 5],
    [5, 1, 1],
    [5, 4, 4],
    [7, 5, 5],
    [4, 5, 5.0000001],
]
```

Ahora la matriz es **numéricamente invertible**: NumPy encuentra una «inversa» aunque en rigor no debería existir, y lo hace con **números gigantes**. Al calcular w , la feature única sale bien, pero las dos columnas casi duplicadas reciben pesos enormes. Eso es lo que vimos en 2.12.

La solución: sumar un número a la diagonal

Si se suma un número pequeño a la **diagonal** de $X^T X$, la columna 3 deja de ser (casi) un duplicado de la columna 2 y la inversa se vuelve estable:

```

XTX = [
    [1, 2, 2],
    [2, 1, 1.0000001],
    [2, 1.0000001, 1],
]
XTX = XTX + 0.01 * np.eye(3)  # suma 0.01 solo en la diagonal

np.linalg.inv(XTX)            # números mucho más pequeños

```

El truco: `np.eye(3)` es la identidad (unos en la diagonal). Sumarla agregaría **1** a la diagonal; multiplicándola por un número pequeño se agrega solo ese número.

Regularización = controlar los pesos para que no crezcan demasiado. Cuanto más grande el número sumado a la diagonal, más pequeños los valores de la inversa y más «bajo control» quedan los pesos.

Implementación

Se copia `train_linear_regression` y se le agrega un parámetro `r` (de *regularization*):

```

def train_linear_regression_reg(X, y, r=0.001):
    ones = np.ones(X.shape[0])
    X = np.column_stack([ones, X])

    XTX = X.T.dot(X)
    XTX = XTX + r * np.eye(XTX.shape[0])

    XTX_inv = np.linalg.inv(XTX)
    w_full = XTX_inv.dot(X.T).dot(y)

    return w_full[0], w_full[1:]

```

Y se reemplaza en el bloque de entrenamiento/validación:

```

X_train = prepare_X(df_train)
w0, w = train_linear_regression_reg(X_train, y_train, r=0.01)

X_val = prepare_X(df_val)
y_pred = w0 + X_val.dot(w)
rmse(y_val, y_pred)

```

El resultado no solo es muchísimo mejor que la versión sin regularizar: también mejora en ~0.5 respecto del mejor modelo anterior (el de 2.11), una mejora considerable.

`r` es un parámetro

Valor de <code>r</code>	Efecto
0	Vuelve a la regresión lineal normal (sin regularización)
Pequeño	Controla los pesos y estabiliza la inversa
Demasiado grande	Al modelo le cuesta aprender → empeora

Hay que encontrar el mejor valor de `r`: eso se hace en la próxima clase.

2.14 — Ajuste del modelo

Vimos que r afecta la calidad del modelo. Ahora se busca **el mejor valor** de r usando el **dataset de validación** (*tuning* de un parámetro).

Probar varios valores de r

Se recorre una lista de valores, desde 0 y aumentando gradualmente, y para cada uno se entrena, se evalúa en validación y se imprime el resultado:

```
for r in [0.0, 0.00001, 0.0001, 0.001, 0.1, 1, 10]:
    X_train = prepare_X(df_train)
    w0, w = train_linear_regression_reg(X_train, y_train, r=r)

    X_val = prepare_X(df_val)
    y_pred = w0 + X_val.dot(w)
    score = rmse(y_val, y_pred)

    print(r, w0, score)
```

Se imprime el parámetro, el **bias** (w_0) y el **score** (RMSE).

Qué se observa

r	Bias w_0	RMSE
0	Enorme	Enorme
Valores pequeños	Razonable	Mejora mucho y casi no cambia entre ellos
Valores grandes ($1, 10$)	Cada vez más chico	Empeora

- Con un poco de regularización ya mejora muchísimo.
- Cuanta más regularización, **más pequeño el bias**.
- Al subir demasiado r el modelo empieza a degradarse.

*Se elige un valor pequeño en el que el modelo **todavía no empieza a degradarse**. Entre los valores pequeños la diferencia es mínima, así que da lo mismo cuál de ellos se elija. En el notebook del curso se usa $r = 0.001$.*

Entrenar con el r elegido

```
r = 0.001
X_train = prepare_X(df_train)
w0, w = train_linear_regression_reg(X_train, y_train, r=r)

X_val = prepare_X(df_val)
y_pred = w0 + X_val.dot(w)
rmse(y_val, y_pred)
```

Funciona bien en validación. Falta comprobarlo en el **dataset de test**, que es lo que se hace en la próxima clase.

2.15 — Usar el modelo

Dos pasos: entrenar el **modelo final** y **usarlo** para predecir el precio de un auto.

Entrenar el modelo final

Hasta ahora se entrenaba en *train* y se evaluaba en *validation*. Para el modelo final:

1. Se entrena con **train + validation** juntos («full train»).
2. Se hace la evaluación final en **test**.

*El RMSE en test **no debería ser muy distinto** del de validación.*

Juntar los DataFrames

`pd.concat` recibe una lista de DataFrames y los concatena:

```
df_full_train = pd.concat([df_train, df_val])
df_full_train = df_full_train.reset_index(drop=True)
```

Sin `reset_index` el índice queda con los valores originales de validación; al resetearlo queda secuencial.

Feature matrix y target

```
X_full_train = prepare_X(df_full_train)
y_full_train = np.concatenate([y_train, y_val])
```

`np.concatenate` hace lo mismo para arrays de NumPy (que no tienen índice, así que no hay nada que resetear).

Entrenar y evaluar en test

```
w0, w = train_linear_regression_reg(X_full_train, y_full_train, r=0.001)

X_test = prepare_X(df_test)
y_pred = w0 + X_test.dot(w)
score = rmse(y_test, y_pred)
score
```

El RMSE en test es prácticamente igual al de validación (hasta el tercer decimal).

*Muy buena señal: el modelo **generaliza bien** y no obtuvo ese score por pura casualidad.*

Usar el modelo para un auto nuevo

La idea: tenemos un auto, extraemos sus features (el *feature vector*), las pasamos al modelo y este predice el precio.

Se toma un auto del test (un Toyota Sienna, con algunos valores faltantes) y se **finge que es nuevo** — es válido porque el modelo nunca lo vio en el entrenamiento.

El auto como diccionario

En la vida real no llega un DataFrame sino algo como un **diccionario**: por ejemplo, una web o app donde el usuario ingresa los datos del auto, que envía un *request* al modelo y recibe el precio de vuelta.

```
car = df_test.iloc[20].to_dict()
car
```

Pasarlo por prepare_X

`prepare_X` espera un **DataFrame**, así que se crea uno con **una sola fila** a partir de una lista con un solo diccionario:

```
df_small = pd.DataFrame([car])
X_small = prepare_X(df_small)
```

Predecir

```
y_pred = w0 + X_small.dot(w)
y_pred = y_pred[0]          # un solo auto → un solo número
y_pred                        # ≈ 10.6
```

El resultado está en **escala logarítmica**. Para volver al precio se deshace el `log1p` con `np.expm1`:

```
np.expm1(y_pred)            # precio predicho
np.expm1(y_test[20])        # precio real
```

La predicción quedó a unos **5.000 dólares** del precio real: una predicción relativamente buena.

Resumen del flujo

```
df_train + df_val —concat—> df_full_train —prepare_X—> X_full_train
                                     |
                                     train_linear_regression_reg(r=0.001)
                                     |
                                     w0, w
                                     |
auto (dict) —pd.DataFrame([car])—> prepare_X —> ŷ = w0 + X·w —np.expm1—> precio
```

2.16 — Resumen de la sesión 2

Repaso de todo el proyecto de predicción del precio de autos.

El proyecto (2.1)

Dataset con precios y características de autos: marca, modelo, motor, tipo de combustible, transmisión, etc. El target es **MSRP** (*manufacturer's suggested retail price*, precio sugerido por el fabricante).

Preparación de datos (2.2)

Se limpió el dataset para que quedara **uniforme**: nombres de columnas y valores con espacios, mayúsculas y minúsculas mezcladas → todo en minúscula y con `_`.

Análisis exploratorio (2.3)

- El precio tenía una distribución de **cola larga** (*long tail*). Se eliminó aplicando la **transformación logarítmica**, porque los modelos de ML suelen tener problemas con colas largas.
- Había **valores faltantes**, con los que no se puede entrenar un modelo: hay que hacer algo con ellos.

Framework de validación (2.4)

Split en **train / validation / test**.

Regresión lineal (2.5–2.6)

Primero para un solo ejemplo, como fórmula con un **for**; después en forma vectorial con **producto punto**, y luego como producto **matriz-vector**. La salida de la regresión lineal son los pesos: el **bias** w_0 y el **vector de pesos** w .

Ecuación normal (2.7)

Cómo obtener los pesos. El ML **no es magia**: es una fórmula, la **ecuación normal**, implementada en NumPy:

$$w = (X^T X)^{-1} X^T y$$

Modelo base (2.8)

Primer modelo con solo **cinco features numéricas** básicas. No anduvo muy bien según el gráfico, pero un gráfico no permite medir el desempeño **de forma objetiva**.

RMSE (2.9–2.10)

Root mean squared error: métrica para evaluar modelos de regresión. Se armó el framework de validación con la función `prepare_X`, que prepara la feature matrix **igual** para train y validation. Bastaba con redefinirla y copiar la misma celda a lo largo de la sesión → **experimentar mucho más rápido**.

Feature engineering (2.11)

Crear features nuevas a partir de las existentes. La feature **age** mejoró el modelo drásticamente: la distribución de las predicciones se parece mucho más a la de los valores reales (no igual, pero mucho mejor).

Variables categóricas (2.12)

Cada variable categórica se representa con un conjunto de **columnas binarias**. Esta codificación se llama **one-hot encoding** (se profundiza en la sesión de clasificación).

El RMSE se volvió enorme de repente. Gracias al dataset de validación el problema se detectó fácilmente.

Regularización (2.13)

La causa era **inestabilidad numérica**. Solución: sumar un número pequeño a la ****diagonal de $X^T X$ **** antes de invertirla:

```
XTX = XTX + r * np.eye(XTX.shape[0])
```

Con las features categóricas y regularización, el modelo mejoró bastante respecto de la versión anterior.

Ajuste del modelo (2.14)

Se probaron distintos valores del parámetro de regularización. Se eligió $r = 0.001$: quizás no es el mejor, pero está al mismo nivel que los otros.

Usar el modelo (2.15)

- Se juntaron train y validation en un **full train** y se entrenó el modelo final (otra vez con `prepare_X`, muy cómodo).
- Se aplicó a un auto del test **fingiendo no conocer su precio**: la predicción no estuvo lejos del precio real.

Paso	Herramienta clave
Limpieza	<code>str.lower()</code> , <code>str.replace(' ', '_')</code>
Target	<code>np.log1p</code> / <code>np.expm1</code>
Split	train / val / test
Entrenamiento	ecuación normal + regularización
Evaluación	RMSE
Features	<code>prepare_X</code> , <code>age</code> , one-hot
Modelo final	<code>pd.concat</code> , <code>np.concatenate</code>

Qué viene después

- Una sección **solo de texto** (sin video) con otras cosas para probar y aprender más sobre el tema.
- La **tarea** (*homework*), para aplicar lo aprendido por cuenta propia.
- La próxima sesión es **clasificación**: en lugar de implementar todo a mano como aquí, se usará **scikit-learn**.

Ya vimos cómo implementar las cosas nosotros mismos; ahora estamos listos para usar la librería.